

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра ІІІ

Звіт

з лабораторної роботи № 7 з дисципліни
«Основи програмування. Частина 2. Модульне програмування»

„ПОБУДОВА ТА ВИКОРИСТАННЯ СТРУКТУР ДАНИХ”

Виконав студент ІІ-56 Гулько Нікіта Вікторович

(шифр, прізвище, ім'я, по батькові)

Перевірів Куценко Микита Олександрович

(прізвище, ім'я, по батькові)

Київ 2026

Мета лабораторної роботи – дослідити типи лінійних та нелінійних структур даних, навчитись користуватись бібліотечними реалізаціями структур даних та будувати власні.

Завдання

9	double	Двоспрямований	Включення до початку	1.Знайти перше входження елемента меншого за середнє значення. 2.Знайти суму елементів, які розташовані після максимального елемента. 3.Отримати новий список зі значень елементів більших за задане значення. 4.Видалити елементи, які розташовані до максимального елемента.
---	--------	----------------	----------------------	--

Виконана робота розміщена на GitHub за посиланням

https://github.com/vv4342nikita-droid/OOP_LAB7

Приклад роботи

```
Початковий список (через foreach): 3 15 20 10 5
-----
Елемент за індексом [2]: 20
Видаляємо елемент №0 (число 3.0)
Список після видалення: 15 20 10 5
-----
ВИКОНАННЯ ЗАВДАНЬ:
1. Перше число, менше за середнє: 10
2. Сума елементів після максимального: 15
3. Елементи більші за 12 (новий список): 20 15
4. Видаляємо всі елементи до максимального...
   Результат: 20 10 5
=====
```

Висновок: Програма працює коректно, поставлене завдання виконане відповідно до вимог.

Відповіді на контрольні запитання:

1. Що таке структура даних?

Це спосіб організації та зберігання інформації в оперативній пам'яті комп'ютера. Залежно від нашої задачі ми обираємо таку структуру, з якою нам буде найзручніше і найшвидше працювати.

2. Наведіть типи структур даних.

Лінійні (масив, список, стек, черга) і нелінійні (дерева, графи, хеш-таблиці).

3. Поясніть, чим відрізняється список від словника.

Список зберігає елементи по порядку. Щоб знайти елемент, ми звертаємося за його номером (індексом).

Словник працює як телефонна книга. Ми шукаємо дані не за порядковим номером, а за унікальним ключем (наприклад, за іменем), щоб отримати його значення (номер телефону).

4. Поясніть, чим відрізняється стек від черги.

Стек — LIFO (останній прийшов — перший вийшов). Черга — FIFO (перший прийшов — перший вийшов).

5. Поясніть сутність побудови дерева.

Дерево — є ієрархічна структура, схожа на файлову систему комп'ютера. Є один головний вузол, від якого йдуть розгалуження до інших вузлів. Це дозволяє дуже швидко знаходити потрібні дані: ми не перебираємо все підряд, а одразу спускаємося потрібною гілкою.

6. Поясніть особливості hash-таблиць.

Її головна особливість — надшвидкий пошук. Коли ви даєте ключ, спеціальна математична формула (хеш-функція) одразу перетворює його на "номерок" (конкретну адресу в пам'яті). Комп'ютеру не треба перебирати весь масив, він за номерком миттєво забирає дані.

7. Поясніть сутність зв'язаної реалізації структур даних.

Зв'язана реалізація — це зберігання елементів у вигляді вузлів, де кожен вузол містить дані та посилання на інші вузли. Пам'ять не обов'язково є суміжною.

8. Наведіть приклад опису типу вузла лінійного списку у зв'язаній реалізації та поясніть устрій вузла.

```
public class Node
{
    public int Data;

    public Node Next;
}
```

Вузол містить дані(Data) і посилання на наступний вузол (Next).

9. Наведіть приклад опису односпрямованого та двоспрямованого списків та поясніть різницю.

Односпрямований:

```
class Node
{ public int Data; public
    Node Next;
}
```

Двоспрямований:

```
class Node
{ public int Data; public
    Node Next; public
    Node Prev;
}
```

Різниця: у двоспрямованому списку є посилання і вперед, і назад, що дозволяє рух у двох напрямках.

10. Наведіть приклади реалізації структур даних у C#.

Масив: `int[] arr = new int[10];`

Список: `List<int>`

Словник: `Dictionary<string, int>`

Стек: `Stack<int>`

Черга: `Queue<int>`